وعدم إعادة فحص المشروع بالكامل، مع AI Agent نعم. سأجعلها **كتلة واحدة فقط** قابلة للنسخ، مع دمج نظام استئناف الـ الحفاظ على البساطة.

POS CLOUD PLATFORM

MASTER ENGINEERING SPECIFICATION v1.1

المواصفات الهندسية الرئيسية لمنصة نقاط البيع السحابية متعددة المنصات

Version / الإصدار: 1.1
Status / الحالة: Architecture Baseline
Languages / اللغات: العربية + English
Development Environment / بيئة التطوير: VS Code
Frontend: Flutter / Dart
Backend: ASP.NET Core / C#
Database: PostgreSQL
Local Database: SQLite
Architecture: Lightweight Modular Monolith
Deployment: Cloud + Offline/Online
Engineering Model: Engineering Cells + Seven Layers
AI Development Model: Multi-Agent with Controlled Continuity

---

1. VISION / الرؤية

العربية

إنشاء منصة POS سحابية حديثة، بسيطة، مستقرة وقابلة للتوسع، تعمل على المنصات الرئيسية، وتخدم المحال التجارية والمطاعم والمخابز والسوبرماركت والصيدليات من خلال نواة برمجية مشتركة.

الهدف هو بناء أساس هندسي قوي، وليس بناء نظام ضخم ومعقد منذ البداية.

يجب أن تكون المنصة:

- سهلة الاستخدام.
- سهلة الصيانة.
- سهلة التطوير.
- قابلة للتوسع.
- Cloud Enabled.

- Offline Capable.
- متعددة المنصات.
- حديثة بصرياً.
- بسيطة هندسياً.
- قابلة للتخصيص بشكل محدود ومدروس.
- قابلة للعمل مع عدة AI Agents بأمان.

English

Build a modern, simple, stable, and extensible Cloud POS platform that supports major platforms and serves retail stores, restaurants, bakeries, supermarkets, and pharmacies through one shared software core.

The objective is to build a strong engineering foundation, not an unnecessarily large or complicated system.

The platform must be:

- Easy to use.
- Easy to maintain.
- Easy to develop.
- Extensible.
- Cloud enabled.
- Offline capable.
- Cross-platform.
- Modern in visual design.
- Simple in architecture.
- Configurable in a controlled manner.
- Safe for multi-agent AI development.

---

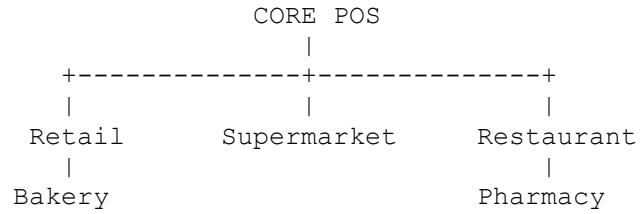
2. CORE BUSINESS TYPES / الأنشطة الأساسية

The platform initially targets:

1. Retail Stores / المحال التجارية
2. Supermarkets / السوبرماركت
3. Restaurants / المطاعم
4. Bakeries / المخابز
5. Pharmacies / الصيدليات

These are not five independent applications.

They share one Core POS.



Business-specific functionality is implemented through Engineering Cells.

---

3. ENGINEERING PHILOSOPHY / الفلسفة الهندسية

The project follows the building principle:

Build the foundation, structural columns, structural walls, roof, and essential services first.

Do not build every possible feature before it is needed.

العربية

نحن نبني:

- الأساس.
- الأعمدة.
- الجدران الحاملة.
- السقف.
- الخدمات الأساسية.

ثم نضيف الغرف والميزات بناءً على حاجة العميل.

English

Build:

- Foundation.
- Structural columns.
- Structural walls.
- Roof.
- Essential services.

Then add features according to real customer requirements.

---

4. GOLDEN PRINCIPLE / المبدأ الذهبي

DESIGN FOR EXTENSION, NOT FOR COMPLEXITY.

العربية

نصمم النظام بحيث يمكن توسيعه مستقبلاً، لكن لا نبني التوسع قبل الحاجة إليه.

English

Design the system so that it can be extended later, but do not implement unnecessary future complexity before it is needed.

---

5. SIMPLICITY RULE / قاعدة البساطة

The simplest solution that satisfies the requirement is preferred.

Do not add:

- unnecessary frameworks;
- unnecessary services;
- unnecessary abstractions;
- unnecessary database tables;
- unnecessary configuration;
- unnecessary APIs;
- unnecessary AI mechanisms.

Readable and maintainable code is more important than theoretical sophistication.

---

6. TECHNOLOGY STACK / التقنيات المعتمدة

```
Frontend:
Flutter / Dart

IDE:
VS Code
```

```
Backend:
ASP.NET Core / C#

Database:
PostgreSQL

Local Database:
SQLite

ORM:
Entity Framework Core

API:
REST API

Authentication:
JWT + Refresh Token

Real-Time:
SignalR only when required

Version Control:
Git

Containerization:
Docker

Deployment:
Cloud
```

The technology stack is an architectural decision.

AI Agents cannot change it independently.

---

7. ARCHITECTURE STYLE / نمط المعمارية

The project uses:

LIGHTWEIGHT MODULAR MONOLITH

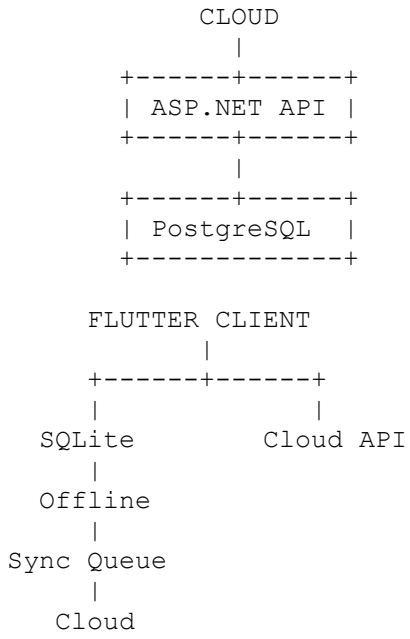
Microservices are NOT part of version 1.

The system should remain a single deployable backend application containing clearly separated logical modules/cells.

Microservices may only be considered later after an explicit architectural review.

---

8. HIGH LEVEL ARCHITECTURE / المعمارية العامة



The POS must continue essential operations when the Internet is temporarily unavailable.

---

9. SEVEN LAYERS / الطبقات السبعة

The architecture uses seven logical layers:

```
L7  Presentation
L6  Application
L5  Domain
L4  Engineering Cells
L3  Infrastructure
L2  Communication
L1  Platform
```

L7 — Presentation

UI, screens, widgets, navigation, themes.

L6 — Application

Use cases and application workflows.

L5 — Domain

Business rules and core business concepts.

L4 — Engineering Cells

Business capability modules.

L3 — Infrastructure

Database, storage, external technical services.

L2 — Communication

REST API, synchronization, external integrations.

L1 — Platform

Operating systems, Cloud, deployment, security infrastructure.

The seven layers are logical boundaries, not a reason to create unnecessary projects or files.

---

10. ENGINEERING CELLS / الخلايا الهندسية

Cells are logical modules.

They are NOT independent applications.

Initial Core Cells:

```
CELL-001 Foundation
CELL-002 Identity
CELL-003 Business
CELL-004 Branch
CELL-005 Product
CELL-006 Inventory
CELL-007 Customer
CELL-008 Supplier
CELL-009 Sales
CELL-010 Payment
CELL-011 Purchasing
```

```
CELL-012 Reporting
```

Business-specific cells:

```
CELL-101 Restaurant
CELL-102 Bakery
CELL-103 Pharmacy
CELL-104 Supermarket
```

Additional cells may be added only when there is a justified business requirement.

---

11. CELL STRUCTURE / هيكل الخلية

Every cell should define:

```
Purpose
Responsibilities
Inputs
Outputs
Business Rules
Database Objects
API Contracts
Algorithms
Tests
Dependencies
```

A cell must not directly manipulate the internal implementation of another cell.

Communication must use clear contracts.

---

12. CORE POS / نواة النظام

The initial Core POS includes:

- Products.
- Categories.
- Barcode.
- Inventory.
- Sales.
- Purchasing.
- Customers.
- Suppliers.
- Payments.

- Users.
- Roles.
- Branches.
- Basic reports.
- Receipts.

Do not add advanced ERP functionality unless required later.

---

13. PAYMENT ENGINE / محرك الدفع

Payment is a Core POS capability.

Initial payment methods:

```
Cash
Card
Electronic Payment
Bank Transfer
Customer Credit
Mixed Payment
Partial Payment
Refund
```

The Payment Engine must remain simple but extensible.

---

14. PAYMENT METHOD VS PROVIDER

Separate:

```
Payment Method
    |
    +-- Cash
    +-- Card
    +-- Transfer
    +-- Electronic
    +-- Credit
```

from:

```
Payment Provider
    |
    +-- External Provider A
    +-- External Provider B
    +-- Future Provider
```

The Sale module must not contain provider-specific payment logic.

Payment integrations must be isolated behind a clear integration boundary.

Sensitive card information must not be stored in the application database unless explicitly required and handled according to applicable security requirements.

---

15. OFFLINE / ONLINE ARCHITECTURE

The POS should continue essential operations during Internet interruption.

```
Flutter POS
     |
   SQLite
     |
 Sync Queue
     |
 Cloud API
     |
PostgreSQL
```

Synchronization states:

```
Pending
Synced
Failed
```

The initial synchronization implementation should remain simple.

Event Sourcing is not required.

---

16. DATABASE PRINCIPLES / مبادئ قاعدة البيانات

Initial core entities:

```
Tenant
Branch
User
Role
Product
Category
Inventory
```

```
Customer
Supplier
Sale
SaleItem
Purchase
PurchaseItem
Payment
```

Database design must prioritize:

- Data integrity.
- Clear relationships.
- Useful indexes.
- Maintainability.
- Performance.
- Auditability.

Do not create unnecessary tables for hypothetical future requirements.

---

17. MULTI-TENANCY / تعدد الشركات

The Cloud platform should support multiple businesses.

Basic model:

```
Tenant
 |
 +-- Branch
      |
      +-- Users
      +-- POS Terminals
      +-- Inventory
      +-- Sales
```

Tenant data must be logically isolated.

---

18. USERS AND ROLES / المستخدمون والصلاحيات

Initial roles:

```
Owner
```

```
Administrator
Manager
Cashier
Inventory User
Accountant
```

The permission system should be simple and capability-based where practical.

Do not build a complex enterprise IAM platform in version 1.

---

19. UI/UX PHILOSOPHY / فلسفة الواجهة

The UI must be:

- Modern.
- Simple.
- Fast.
- Clear.
- Responsive.
- Touch friendly.
- Desktop friendly.
- Mobile friendly.
- Visually attractive.

The design objective is:

Modern simplicity that makes the customer feel the product is professional.

The interface must not become complicated merely to look advanced.

---

20. DESIGN SYSTEM / نظام التصميم

Initial Design System:

```
Colors
Typography
Spacing
Buttons
Cards
Forms
Dialogs
Tables
Navigation
```

```
Icons
```

All screens must use the same Design System.

Do not create a different design language for every screen.

---

21. ADAPTIVE UI / الواجهة القابلة للتكيف

The system should eventually allow controlled customization through an administration console.

Initial configuration:

```
Logo
Business Name
Primary Color
Secondary Color
Light/Dark Mode
Language
Currency
Receipt Template
Basic Dashboard Layout
```

Do NOT build a full dynamic UI builder in version 1.

If customers later require additional customization, add it incrementally.

---

22. BUSINESS-SPECIFIC CELLS / الخلايا المتخصصة

Restaurant Cell

Potential features:

```
Tables
Orders
Kitchen
Kitchen Display
Modifiers
Reservations
```

Bakery Cell

Potential features:

```
Recipes
Production
Batches
Production Cost
Expiry
```

Pharmacy Cell

Potential features:

```
Batches
Expiry
Prescriptions
Medicine-specific information
```

Supermarket Cell

Potential features:

```
Barcode
Scale
Promotions
Large Inventory
Fast Checkout
```

These features are extensions, not separate applications.

---

23. API PRINCIPLES / مبادئ API

Use a simple REST API.

Examples:

```
/api/products
/api/sales
/api/customers
/api/inventory
/api/purchases
/api/payments
```

Every API endpoint must define:

```
Purpose
Authentication
```

```
Authorization
Request
Response
Validation
Errors
Business Rules
```

Avoid unnecessary API abstraction.

---

24. CODE SIMPLICITY / بساطة الكود

Use:

Simple First.

Do not create abstractions without a real reason.

Avoid automatically creating:

```
GenericRepository
GenericService
GenericManager
GenericFactory
Provider
Adapter
Wrapper
```

unless the abstraction solves an actual problem.

A small amount of controlled duplication may be preferable to excessive abstraction.

---

25. CLEAN ARCHITECTURE / العمارة النظيفة

Use Clean Architecture principles pragmatically.

Do not create 10–15 files for a simple business operation merely to satisfy a pattern.

The goal is:

```
Clear responsibility
+
Testability
+
```

```
Maintainability
```

not maximum abstraction.

---

26. DEVELOPMENT WORKFLOW / دورة التطوير

The standard workflow is:

```
Understand
    ↓
Design
    ↓
Document
    ↓
Implement
    ↓
Test
    ↓
Review
    ↓
Integrate
    ↓
Checkpoint
```

Never use:

```
Generate huge code
        ↓
Try to understand it later
```

---

27. MULTI-AGENT DEVELOPMENT / التطوير باستخدام عدة وكلاء

The project may use multiple AI Agents.

Example:

```
Agent A
Architecture / Planning

Agent B
Flutter / UI
```

```
Agent C
ASP.NET Core / Backend

Agent D
Database

Agent E
Testing / QA

Agent F
Security
```

All agents must use the same Master Specification.

No individual agent owns the architecture.

---

28. MASTER SOURCE OF TRUTH / المرجع الأساسي

The official project documentation consists of:

```
00_MASTER_SPECIFICATION.md
01_ARCHITECTURE.md
02_ENGINEERING_CELLS.md
03_DATABASE.md
04_API_SPECIFICATION.md
05_ALGORITHMS.md
06_UI_UX.md
07_BUSINESS_CELLS.md
08_TESTING.md
09_IMPLEMENTATION_PLAN.md
AI_AGENT_RULES.md
PROJECT_STATE.md
```

These documents are the official source of truth.

Generated code must follow them.

---

29. AI AGENT GOVERNANCE / حوكمة الوكلاء

Every AI Agent must:

1. Read the relevant project rules.

2. Identify the current phase.
3. Identify the current cell.
4. Identify the current task.
5. Work only within the approved scope.
6. Avoid unnecessary changes.
7. Run relevant tests.
8. Update the project state.
9. Create a checkpoint after meaningful progress.

Agents must not independently redesign the system.

---

30. ARCHITECTURAL CHANGE CONTROL / التحكم بالتغييرات المعمارية

Change levels:

```
L0
Simple code change

L1
Bug fix

L2
Internal cell change

L3
Compatible API/contract change

L4
Cross-cell architectural change

L5
Core database change

L6
Seven-layer change

L7
Architecture or technology-stack change
```

L0-L3 may normally proceed within the approved design.

L4-L7 require Architectural Review.

---

31. ARCHITECTURAL CONFLICT RULE / قاعدة التعارض المعماري

If an Agent discovers that completing the requested task requires:

- changing the architecture;
- changing the database foundation;
- changing the technology stack;
- changing cell boundaries;
- changing the seven-layer model;
- introducing a major new framework;
- introducing Microservices;
- removing an architectural principle;

the Agent MUST:

```
STOP
```

The Agent must NOT make the change.

The Agent must NOT ask the user for a quick approval such as:

"Can I change the architecture?"

Instead, create:

```
ARCHITECTURAL_CONFLICT_REPORT.md
```

containing:

```
Problem
Current Architecture
Required Change
Why the Current Design Is Insufficient
Affected Components
Risks
Possible Alternatives
Recommended Solution
Status = BLOCKED
```

The change remains blocked until Architectural Review.

---

32. AI AGENT CONTINUITY SYSTEM / نظام استمرارية الوكيل

The project must not depend on an AI Agent remembering the project.

The project itself stores the current development state.

Primary file:

```
PROJECT_STATE.md
```

This file is the operational memory of the project.

---

33. PROJECT_STATE.md / حالة المشروع

The file should contain:

```
Project
Current Phase
Current Cell
Current Task
Completed Work
In Progress
Next Task
Known Issues
Blocked Items
Last Checkpoint
Tests
Architecture Status
Last Updated
```

Example:

```
Project:
POS Cloud Platform

Current Phase:
PHASE-04

Current Cell:
CELL-005 Product

Current Task:
Implement Product API

Completed:
```

```
Product Entity
Database Migration
Repository

In Progress:
Product Service

Next:
Product Controller
Validation
Tests

Known Issues:
None

Blocked:
None

Last Checkpoint:
CP-004

Architecture Status:
UNCHANGED
```

---

34. CHECKPOINT SYSTEM / نظام نقاط التوقف

A checkpoint is created after meaningful work.

Example:

```
CP-001
Foundation completed

CP-002
Identity completed

CP-003
Database baseline completed

CP-004
Product Entity completed
```

Each checkpoint should record:

```
Checkpoint ID
Date
Phase
Cell
Task
Files Changed
Tests
Result
```

```
Next Step
Architecture Status
```

---

35. TASK RESUMPTION PROTOCOL / بروتوكول استئناف المهمة

When an Agent resumes an interrupted task:

The Agent MUST:

1. Read `PROJECT_STATE.md`.
2. Read the latest checkpoint.
3. Read only the documentation relevant to the current task.
4. Read only the files directly related to the current cell.
5. Check Git status/diff when applicable.
6. Continue from the last checkpoint.
7. Run only relevant tests first.
8. Update the checkpoint.
9. Update `PROJECT_STATE.md`.

The Agent MUST NOT:

- scan the entire project unnecessarily;
- reread all documentation;
- redesign the architecture;
- re-evaluate unrelated code;
- inspect unrelated cells;
- regenerate already completed code.

---

36. NO FULL PROJECT REVIEW ON RESUME / منع المراجعة الشاملة عند الاستئناف

Default rule:

RESUME, DO NOT RESTART.

If the task was interrupted because of:

- Internet outage;
- VS Code restart;

- Agent restart;
- context limit;
- session termination;
- temporary error;

the Agent should resume from the saved state.

A full project review is allowed only when:

1. The task explicitly requires it.
2. The project state is inconsistent.
3. A serious integrity problem is detected.
4. An architectural conflict is detected.
5. The user explicitly requests a full review.

---

37. MINIMAL CONTEXT LOADING / تحميل الحد الأدنى من السياق

For every task, the Agent should load:

```
Master Specification
+
Relevant Architecture Section
+
Current Cell Specification
+
PROJECT_STATE.md
+
Relevant Source Files
```

Do not load unrelated documents or source code.

This is required to reduce:

- Token usage.
- Processing time.
- Context confusion.
- Accidental changes.

---

38. GIT CHECKPOINTS / نقاط Git

Use Git as a technical recovery mechanism.

After completing a meaningful unit:

```
git status
git diff
tests
commit
checkpoint
```

Suggested commit style:

```
feat(products): add product entity

fix(inventory): correct stock deduction

test(sales): add sale calculation tests

docs(state): update project checkpoint
```

Never commit known broken critical code as a completed feature.

---

39. AGENT HANDOFF / تسليم المهمة بين الوكلاء

When Agent A finishes and Agent B continues:

Agent A must update:

```
PROJECT_STATE.md
```

and create a checkpoint.

The next Agent reads the state instead of re-analyzing the entire project.

Example:

```
Flutter Agent
      |
      ▼
Checkpoint
      |
      ▼
PROJECT_STATE.md
      |
      ▼
Backend Agent
```

---

40. AGENT FAILURE RECOVERY / استرداد الأخطاء

If an Agent fails during a task:

1. Preserve the current code.
2. Record the failure.
3. Update `PROJECT_STATE.md`.
4. Do not restart from zero.
5. Do not rewrite unrelated code.
6. Continue from the last valid checkpoint.
7. If the failure indicates architectural incompatibility, stop and create an Architectural Conflict Report.

---

41. INTERNET INTERRUPTION / انقطاع الإنترنت

When Internet access is interrupted:

The development environment should preserve:

- Current files.
- Git state.
- PROJECT_STATE.md.
- Checkpoints.
- Local database where applicable.

When the Agent reconnects:

```
Read PROJECT_STATE.md
        ↓
Read Last Checkpoint
        ↓
Identify Current Task
        ↓
Read Relevant Files
        ↓
Continue
```

Do not perform a full project review automatically.

Important limitation:

Cloud-based AI generation requires Internet access unless a locally hosted model is available.

The continuity system preserves project state; it does not magically provide cloud AI when the network is unavailable.

---

42. TOKEN EFFICIENCY / كفاءة استهلاك التوكنز

The system is explicitly designed to minimize AI context usage.

Rules:

```
Do not repeat the full specification.
Do not read unrelated files.
Do not regenerate existing code.
Do not explain unchanged modules repeatedly.
Use PROJECT_STATE.md.
Use checkpoints.
Use focused tasks.
Use focused agents.
```

Each AI session should have a clearly bounded objective.

---

43. AI TASK FORMAT / صيغة مهمة الوكيل

Every AI task should be structured as:

```
TASK ID:
CELL:
PHASE:
OBJECTIVE:
ALLOWED FILES:
RELEVANT DOCUMENTS:
EXPECTED RESULT:
TEST REQUIREMENTS:
ARCHITECTURAL RESTRICTIONS:
NEXT CHECKPOINT:
```

Example:

```
TASK ID:
PROD-API-001

CELL:
CELL-005 Product
```

```
PHASE:
PHASE-04

OBJECTIVE:
Implement Product API

ALLOWED FILES:
Product module only

RELEVANT DOCUMENTS:
Master Specification
Product Cell Specification
API Specification

EXPECTED RESULT:
CRUD Product API

TEST REQUIREMENTS:
API validation tests

ARCHITECTURAL RESTRICTIONS:
No architecture changes

NEXT CHECKPOINT:
CP-005
```

---

44. DEFINITION OF DONE / تعريف إنجاز المهمة

A task is complete only when:

```
Implementation complete
+
Tests complete
+
No critical known errors
+
Documentation updated
+
PROJECT_STATE updated
+
Checkpoint created
+
Architecture unchanged
```

---

45. DATABASE SIMPLICITY / بساطة قاعدة البيانات

Do not design an enormous ERP database.

Start with the minimum business model.

Expand only when business requirements justify it.

---

46. REPORTING / التقارير

Initial reports should be limited to essential operational reports:

```
Daily Sales
Sales by Period
Product Sales
Inventory
Purchases
Payments
Cashier Summary
Basic Profit Information
```

Advanced BI and analytics can be added later.

---

47. ANALYTICS / تحليل البيانات

The database should preserve clean transactional data so that advanced analytics can be added later.

Do not build a separate Data Warehouse in version 1 unless a real requirement exists.

The system should keep transactional data normalized and auditable.

---

48. SECURITY / الأمان

Minimum requirements:

```
Authentication
Authorization
Password Hashing
JWT
Refresh Tokens
Tenant Isolation
Input Validation
HTTPS
```

```
Audit Logging
Secure Secrets
Backup
```

Security must be part of the foundation.

---

49. AUDIT / التدقيق

Audit important operations:

```
Login
Logout
Sale
Refund
Payment
Inventory Adjustment
Price Change
Permission Change
Configuration Change
```

Do not log passwords, tokens, card secrets, or sensitive credentials.

---

50. HARDWARE / العتاد

The architecture should allow future integration with:

```
Barcode Scanner
Receipt Printer
Cash Drawer
POS Terminal
Customer Display
Scale
Kitchen Printer
Kitchen Display
```

Hardware-specific code must remain isolated from core business logic.

---

51. BUSINESS EXTENSION RULE / قاعدة إضافة وظائف العميل

When a customer requests a new feature:

First determine:

```
Which cell owns the feature?
Does it already exist?
Can the existing design support it?
Is a new cell required?
Does it affect the architecture?
```

If it can be added inside an existing cell:

Add it there.

Do not redesign the system.

---

52. IMPLEMENTATION PHASES / مراحل التنفيذ

```
PHASE-00
Architecture & Documentation

PHASE-01
Foundation

PHASE-02
Identity

PHASE-03
Business + Branch

PHASE-04
Products

PHASE-05
Inventory

PHASE-06
POS

PHASE-07
Sales + Payments

PHASE-08
Offline + Synchronization

PHASE-09
Purchasing
```

```
PHASE-10
Customers + Suppliers

PHASE-11
Reports

PHASE-12
Restaurant Cell

PHASE-13
Bakery Cell

PHASE-14
Pharmacy Cell

PHASE-15
Supermarket Cell

PHASE-16
Testing + Hardening

PHASE-17
Deployment
```

Each phase is completed and checkpointed before moving to the next major phase.

---

53. DOCUMENTATION STRUCTURE / هيكل الوثائق

The project documentation is:

```
docs/
│
├── 00_MASTER_SPECIFICATION.md
├── 01_ARCHITECTURE.md
├── 02_ENGINEERING_CELLS.md
├── 03_DATABASE.md
├── 04_API_SPECIFICATION.md
├── 05_ALGORITHMS.md
├── 06_UI_UX.md
├── 07_BUSINESS_CELLS.md
├── 08_TESTING.md
├── 09_IMPLEMENTATION_PLAN.md
├── AI_AGENT_RULES.md
├── PROJECT_STATE.md
└── ARCHITECTURAL_CONFLICT_REPORT.md
```

---

54. PROJECT STRUCTURE / هيكل المشروع

```
POS/
├── docs/
├── frontend/
│   └── Flutter
├── backend/
│   └── ASP.NET Core
├── database/
├── tests/
└── deployment/
```

Keep the physical project structure simple.

---

55. ARCHITECTURAL DECISIONS / القرارات المعمارية

ADR-001

Flutter + ASP.NET Core + PostgreSQL + SQLite.

Reason:

Cross-platform capability, maintainability, strong ecosystem, and alignment with the development environment.

ADR-002

Lightweight Modular Monolith.

Reason:

Avoid unnecessary Microservices complexity.

ADR-003

Engineering Cells are logical modules.

Reason:

Provide modularity without creating independent applications.

ADR-004

Offline-capable POS.

Reason:

Retail operations must continue during temporary Internet interruptions.

ADR-005

Payment Engine is part of Core POS.

Reason:

Payments are fundamental to POS.

ADR-006

Limited UI customization initially.

Reason:

Maintain simplicity while supporting customer branding.

ADR-007

AI Agents cannot independently change architecture.

Reason:

Prevent architectural drift and accidental changes.

ADR-008

PROJECT_STATE.md + Checkpoints.

Reason:

Allow multiple AI Agents to resume work without rereading the entire project.

ADR-009

Minimal Context Loading.

Reason:

Reduce token consumption, execution time, and context confusion.

---

56. MASTER AI AGENT RULES / القواعد الرئيسية لوكلاء الذكاء الاصطناعي

Every AI Agent must follow these rules:

```
1. Read the specification.
2. Read PROJECT_STATE.md.
3. Identify the current phase.
4. Identify the current cell.
5. Identify the current task.
6. Load only relevant documentation.
7. Load only relevant source files.
8. Do not perform an unnecessary full-project review.
9. Do not redesign the architecture.
10. Do not add unnecessary complexity.
11. Do not introduce new frameworks without review.
12. Do not introduce Microservices.
13. Do not change the database foundation without review.
14. Do not change cell boundaries without review.
15. Do not change the seven-layer architecture.
16. Implement only the requested task.
17. Test the implementation.
```

```
18. Update PROJECT_STATE.md.

19. Create a checkpoint.

20. If an architectural conflict appears:
    STOP.
    Create ARCHITECTURAL_CONFLICT_REPORT.md.
    Do not ask for quick approval.
    Wait for architectural review.
```

---

57. MASTER RULE / القاعدة العليا

THE AI AGENT IS AN IMPLEMENTER, NOT THE OWNER OF THE ARCHITECTURE.

العربية

الـ AI Agent هو منفذ ومساعد هندسي.

ليس صاحب القرار النهائي في المعمارية.

---

58. ARCHITECTURAL SAFETY / الحماية المعمارية

No AI Agent may silently:

```
Change database architecture
Change framework
Change technology
Change layer
Change cell boundaries
Introduce Microservices
Remove core security
Remove synchronization rules
Change tenancy model
```

Such changes require formal architectural review.

---

59. MAINTAINABILITY / قابلية الصيانة

The project must be understandable by a developer who joins the project later.

A future developer should be able to understand:

```
What the system does
How it is structured
Where a feature belongs
How data flows
How payments work
How synchronization works
How agents should work
```

without reverse-engineering the entire application.

---

60. FINAL ENGINEERING PRINCIPLES / المبادئ النهائية

```
1. Simple First
2. Build the Foundation First
3. Modular Monolith
4. Cells as Logical Boundaries
5. Seven Layers, Lightweight
6. Reuse Before Abstraction
7. Offline Capability
8. Payment as Core
9. UI Modern but Simple
10. Configuration Before Dynamic UI
11. No Premature Features
12. No Premature Microservices
13. Documentation Before Large Implementation
14. Checkpoint Every Meaningful Unit
15. Resume Instead of Restart
16. Minimal Context Loading
17. AI Agents Follow the Specification
18. AI Agents Cannot Change Architecture
19. Architectural Conflicts Must Stop Execution
20. Extend the System Without Destroying Its Foundation
```

---

61. CURRENT PROJECT STATE / حالة المشروع الحالية

```
Project:
POS Cloud Platform

Version:
```

```
1.1

Architecture:
Approved Baseline

Development Status:
Documentation / Architecture

Current Phase:
PHASE-00

Current Cell:
None

Current Task:
Architecture Baseline

Completed:
- Project Vision
- Core Business Model
- Technology Stack
- Seven Layers
- Engineering Cells
- Payment Engine
- Offline Architecture
- UI/UX Principles
- Multi-Agent Model
- AI Agent Governance
- Architectural Change Control
- PROJECT_STATE
- Checkpoint System
- Task Resumption Protocol
- Minimal Context Loading
- Git Checkpoint Strategy

Next:
Architecture Diagrams
ERD
Cell Specifications
API Specification
Algorithms
UI/UX Specification
Implementation Plan

Architecture Status:
UNCHANGED

Blocked:
None
```

---

62. FINAL PROJECT STATEMENT / البيان النهائي للمشروع

العربية

هذا المشروع ليس مجرد برنامج نقاط بيع.

إنه منصة POS سحابية متعددة المنصات، مبنية على أساس هندسي بسيط وقابل للتوسع، تستخدم الخلايا الهندسية والطبقات السبعة لتنظيم النظام دون تحويله إلى بنية معقدة.

يتم بناء الأساس أولاً، ثم تضاف الميزات والخلايا المتخصصة حسب الحاجة الفعلية للعملاء.

ويتم استخدام عدة AI Agents ونظام Checkpoints لتسريع التطوير، لكن جميع الوكلاء يخضعون لمرجع هندسي واحد، وحالة مشروع مشتركة، وقواعد صارمة تمنع التغييرات المعمارية غير المصرح بها.

يتم استئناف المهمة من آخر حالة محفوظة بدلاً من إعادة تحليل المشروع بالكاملAgent عند انقطاع الاتصال أو توقف.

English

This project is not merely a Point of Sale application.

It is a cross-platform Cloud POS platform built on a simple and extensible engineering foundation, using Engineering Cells and Seven Logical Layers to organize the system without turning it into unnecessary complexity.

The foundation is built first, and specialized features are added according to real customer requirements.

Multiple AI Agents may be used to accelerate development, but all agents operate under one engineering source of truth, a shared project state, checkpoints, and strict rules preventing unauthorized architectural changes.

When connectivity is interrupted or an Agent stops, the task resumes from the last saved state instead of re-analyzing the entire project.

END OF MASTER ENGINEERING SPECIFICATION v1.1

POS Cloud Platform

Foundation First — Simple, Maintainable, Extensible

الأساس أولاً ـــ البساطة ـــ قابلية الصيانة ـــ قابلية التوسع